

Manuscript version: Author's Accepted Manuscript

The version presented in WRAP is the author's accepted manuscript and may differ from the published version or Version of Record.

Persistent WRAP URL:

<http://wrap.warwick.ac.uk/187403>

How to cite:

The repository item page linked to above, will contain details on accessing citation guidance from the publisher.

Copyright and reuse:

The Warwick Research Archive Portal (WRAP) makes this work of researchers of the University of Warwick available open access under the following conditions.

This article is made available under the Creative Commons Attribution 4.0 International license (CC BY 4.0) and may be reused according to the conditions of the license. For more details see: <http://creativecommons.org/licenses/by/4.0/>.



Publisher's statement:

Please refer to the repository item page, publisher's statement section, for further information.

For more information, please contact the WRAP Team at: wrap@warwick.ac.uk

Path-following control of unmanned underwater vehicle based on an improved TD3 deep reinforcement learning

Yexin Fan, Hongyang Dong, *Member, IEEE*, Xiaowei Zhao, *Member, IEEE*, Petr Denissenko

Abstract—This work proposes an innovative path-following control method, anchored in deep reinforcement learning (DRL), for unmanned underwater vehicles (UUV). This approach is driven by several new designs, all of which aim to enhance learning efficiency and effectiveness and achieve high-performance UUV control. Specifically, a novel experience replay strategy is designed and integrated within the **twin-delayed deep deterministic policy gradient algorithm (TD3)**. It distinguishes the significance of stored transitions by making a trade-off between **rewards and temporal-difference (TD) errors**, thus enabling the UUV agent to explore optimal control policies more efficiently. Another major challenge within this control problem arises from action oscillations associated with DRL policies. This issue leads to excessive system wear on actuators and makes real-time application difficult. To mitigate this challenge, a newly improved regularization method is proposed, which provides a moderate level of smoothness to the control policy. Furthermore, a dynamic reward function featuring adaptive constraints is designed to avoid unproductive exploration and expedite learning convergence speed further. Simulation results show that our method garners higher rewards in fewer training episodes compared to mainstream DRL-based control approaches (e.g., **Deep Deterministic Policy Gradient (DDPG)** and vanilla TD3) in UUV applications. Moreover, it can adapt to varying path configurations amid uncertainties and disturbances, all while ensuring high tracking accuracy. **Simulation and experimental studies are conducted to verify the effectiveness.**

Index Terms—Deep reinforcement learning, unmanned underwater vehicle, path-following control, intelligent control

I. INTRODUCTION

IN recent years, unmanned underwater vehicles (UUVs), typically including autonomous underwater vehicles (AUVs) and remotely operated underwater vehicles, have become increasingly popular in the fields of marine resource exploration and ocean science research. Due to their impressive maneuverability and versatility, UUVs have been extensively used in a variety of underwater tasks, including pipeline tracking [1], seafloor mapping [2], underwater structure inspections [3], and military operations [4]. To accomplish these tasks effectively, accurate path-following control needs to be guaranteed [5].

This work has received funding from the UK Engineering and Physical Sciences Research Council under Grant EP/S001905/1.

Y. Fan, H. Dong, and X. Zhao (corresponding author) are with the Intelligent Control and Smart Energy (ICSE) Research Group, School of Engineering, University of Warwick, Coventry, CV4 7AL, United Kingdom. Emails: yexin.fan@warwick.ac.uk, hongyang.dong@warwick.ac.uk, xiaowei.zhao@warwick.ac.uk

P. Denissenko is with the School of Engineering, University of Warwick, Coventry, CV4 7AL, United Kingdom. Email: P.Denissenko@warwick.ac.uk

Considerable research and studies have been devoted to the study of path-following problems. Traditional control methods, such as SMC [6], MPC [7, 8], and adaptive control [9] have been well-studied to solve these problems. The SMC method can effectively deal with uncertain dynamics and external disturbances, while the high oscillation control frequency around the sliding surface causes increased energy consumption and brings complexion to practical implementation. Adaptive control is a feedback method that aims to automatically adjust the controller parameters based on the changing dynamics or uncertainties. However, the stability of the system may not be assured with the increased system complexion. The MPC method is an optimal control method that utilizes the system's dynamic model to predict its future behavior and optimize the control inputs over a finite time horizon. However, this method struggles with system uncertainties and long time prediction horizons. In addition, a common drawback among the aforementioned methods is that they all heavily rely on the systems' analytical modeling, which can be hard to obtain or inaccurate in practice, potentially leading to significant performance degradation in real applications.

To avoid the limitation of model-based controllers, researchers turn their attention to model-free methods. Proportional-integral-derivative (PID) [10, 11, 12] is one of the most widely used model-free control algorithms in UUV practical applications, due to its simple control structure and easy implementation characteristic. However, tuning a set of optimal PID parameters that accommodates different operating scenarios is time-consuming and labor-intensive. In addition, its control performance can be degraded by the highly nonlinear hydrodynamics and external disturbances [13, 14].

Reinforcement learning (RL) has emerged as a cutting-edge and efficient data-driven technique in recent years, owing to the rapid development of artificial intelligence (AI). By combining machine learning and optimization techniques, **RL has led to wide adoption in both scientific research and industrial applications**, [15, 16, 17, 18, 19]. One of the key ideas behind RL is that an agent learns to improve its control policy by maximizing the accumulated reward through interactions with the environment. This representative feature allows RL algorithms to handle systems with unknown dynamics, thereby enabling UUVs to acquire optimal control policies that can function efficiently in complex and dynamic environments. Various RL control schemes have been applied to the UUV posture and path-following control [20, 21, 22, 23, 24]. For example, in [20], a DDPG-based control algorithm is applied

to realize the posture control of a 5-DOF AUV under low-level controllers, where raw sensory information is directly taken as the input and output of the thruster commands. A real experiment result shows that the controller has the ability to reach the desired velocity. In [21], a generative adversarial imitation learning algorithm that learns from the demonstrations and additional human rewards is proposed for AUV path-following. Though the controller trained with this method achieves faster learning speed, it has limited generalization ability which can only be adapted to simple path configuration. In [22], a fast-deployed DDPG method is developed to serve the same purpose for a six-degree-of-freedom X-rudder AUV. However, the influences of external disturbances are not considered in the simulation. Since the conventional DDPG algorithm suffers from the problem of overestimation and high model variance, the study in [25] proposes a twin-delayed DDPG (TD3) method to compensate for these defects. In [24], the TD3 algorithm is applied to the AUV docking control. Simulation results indicate that the TD3 algorithm achieves more efficient learning than other popular RL algorithms.

During the RL learning process, to ensure that the controller trained in a simulated environment can be effectively transferred to real-world scenarios, these environments are typically augmented with modeling uncertainties and external disturbances. Though these factors can help mitigate the gap between the simulation and the practical application to improve the controller's robustness, they also lead to learning complexity and reduced training efficiency. Several studies have endeavored to tackle these problems. For instance, in [26], the prioritized experience replay (PER) is employed to achieve faster learning speed. In [27], a modified DDPG algorithm is proposed by utilizing an optimized sample pooling and average motion critic networks to design path-following controllers for AUVs. The simulation verified that its performance is better than the S-plane controller under the presence of modeling uncertainties and external disturbances. However, this method adopts six critic networks in the **neural network(NN)** structure, which requires considerable computation resources. In [28], a DDPG-based controller is designed for AUV trajectory tracking and obstacle avoidance. During the training process, a safety constraint is incorporated into the reward function to achieve faster convergence. However, the formulation of this reward design cannot fully reflect its tracking effectiveness. In [24], a TD3 controller combined with imitation learning is proposed for the UUV motion control. Though the results illustrate that the proposed method can improve the convergence speed and is more stable than the other RL methods, the imitation learning method relies on a large number of expert demonstrations, which may be impossible to obtain under some specific situations.

Furthermore, since RL network structures are of high complexity, the control policies output by the actor network may experience excessive oscillations. Such kind of oscillations may increase energy consumption and shorten the UUV's actuator lifespan, making it less practical for real-world applications [29, 30]. Some studies propose using a penalty function related to action constraints and incorporating it into

the reward design to mitigate this issue. In [31, 32], a penalty function is designed based on the standard deviation of a fixed number of historical thrust command values. However, this method requires careful parameter tuning and may not guarantee the agent to learn the desired behavior [33]. In [34], a smoothing strategy is proposed for the control of high-speed drifting cars. In this method, the policies' output is directly smoothed by the current action and the previous action with their corresponding assigned weights. However, this strategy changes the dynamical response expected by the RL network and violates the Markov assumption, which may lead to learning failure.

To address the above issues, this paper develops a novel path-following control method for UUVs via deep reinforcement learning (DRL). The proposed method is built upon TD3 with improved learning efficiency and effectiveness, and we denote it as TD3-IMP. The primary contribution of this work is as follows.

(1) This paper develops an enhanced TD3 reinforcement learning method for UUV path-following control. Our method is robust under uncertainties and disturbances and can adapt to a range of path configurations. As a model-free and data-driven method, it largely circumvents the performance degradation often associated with traditional model-based control methods, which can be hindered by unavoidable modeling errors and uncertainties. Experimental studies are conducted to verify the effectiveness.

(2) A novel experience replay mechanism, which combines the sampling criterion of TD-error and the averaged episode reward is designed to offer more vital and informative experiences for neural network training. Compared with the traditional random sampling mechanism and PER method[22, 26, 28] employed in UUV training, this technique significantly improves the sampling efficiency of TD3, which thereby accelerates the convergence speed of the learning process without compromising its effectiveness.

(3) A refined policy regulation formation is introduced to smooth the actions learned by DRL. Unlike the method proposed in [35], where the smoothness parameter is strictly fixed, our proposed formulation adaptively adjusts the smoothness strength according to the UUV's tracking error, which offers a notable enhancement in eliminating the action oscillation while guaranteeing overall path-following performance.

(4) To alleviate computation burdens and complexities, a dynamic reward function is designed to guide the vehicle in the early stages of training. This function accelerates convergence for DRL agents and prevents them from becoming ensnared in local optima.

The remainder of this paper is summarized as follows. In Section II, the UUV dynamics used in this work are presented. In Sections III&IV, a path-following controller designed based on the proposed improved TD3 reinforcement learning is introduced. In Section V, the path-following simulation results are discussed and compared with various advanced RL methods, and experiment results are given. Section VI gives the conclusion of this paper.

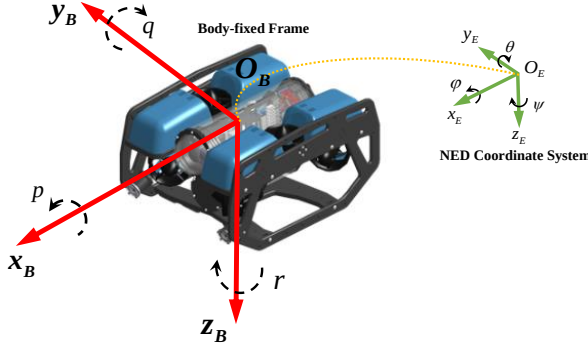


Fig. 1. The coordinate system of the underwater vehicle. NED coordinate system and the body-fixed frame.

II. PRELIMINARIES

A. The UUV kinematic and dynamic model

The mathematical modeling of UUVs is commonly defined in two coordinate frames, which are the NED (North-East-Down) coordinate system O_E and the body-fixed frame O_B , as shown in Fig. 1. According to SNAME [36], six independent coordinates are employed to determine the vehicle's position and orientation [37]. In O_E , the x_E , y_E , and z_E axis points the true north, the east, and downwards normal to the Earth's surface, respectively. While in O_B , x_B and y_B point to the front and the side of the body, respectively, and z_B axis points to the same direction of z_E . The dynamic modeling of the UUV is used to compute the relationship between the desired motion of the UUV and the forces executed by actuators, which is expressed as

$$\begin{aligned} M\dot{v} + C(v)v + D(v)v + g(\eta) &= \tau + \Delta\tau, \\ \dot{\eta} &= J(\eta)v, \end{aligned} \quad (1)$$

where $v = [u, v, w, p, q, r]^T$ is the linear and angular velocities vector of the UUV with respect to O_B , $M \in R^{6 \times 6}$ denotes the inertial matrix, $C(v) \in R^{6 \times 6}$ describes Coriolis matrix, $D(v) \in R^{6 \times 6}$ defines the hydrodynamic damping matrix, $g(\eta) \in R^{6 \times 6}$ is the vector of hydro-static forces, $\tau = [X, Y, Z, K, M, N]^T \in R^{6 \times 6}$ represents the vector of external forces and moments in the body-fixed frame, $\Delta\tau \in R^{6 \times 6}$ denotes the unknown time-varying disturbance vector, $\eta = [x, y, z, \phi, \theta, \psi]^T$ represents the vehicle's position and attitude state vector with respect to O_E , and $J(\eta)$ is the transformation matrix between O_B and O_E [37]. It's important to note that the control method we propose in this paper is data-driven and model-free. The models introduced here are solely for illustrative purposes and serve as the foundational baseline in the design of UUV simulator.

Here we consider a commercial UUV, BlueROV, and adapt the general model in Eq. (1) to the specifications of BlueROV2. As depicted in Fig. 1, the buoyancy center of BlueROV is carefully placed to maintain mechanical stability in two degrees of freedom (DOFs) (i.e., $\phi = \theta \approx 0$). Thus in this paper, we focus on the path-following control with 4-DOFs (i.e., surge, sway, heave, yaw).

In practical experimentation, the motion control of UUV is powered by thrusters. Four thrusters, arranged on a horizontal

plane, deliver propulsion for surge, sway, and yaw within the body-fixed frame. Additionally, two vertically mounted thrusters cater to heave motion control. As in BlueROV, UUV operates by converting thruster control signal commands into pulse-width modulation (PWM) signals, which are then transmitted to the Electronic Speed Controller (ESC) modules of each respective thruster. Generally, the relationship between thruster commands and its generated force and moment exhibits nonlinearity. Here, for the sake of simplification, we adopt a linear approximation that has been verified by experimental trials [38, 39]. This can be represented by the equation $\tau = HT$, where H is the thrust configuration matrix, depending on the technical specification of the UUV [40], and T is the matrix of thruster commands. The benchmarking hydrodynamic parameters of BlueROV are referenced in [39, 40].

III. UUV PATH-FOLLOWING CONTROL USING RL

In this study, we apply RL to address the UUV path-following problem. RL is a framework that enables an agent, in this case, the UUV, to acquire a control policy through interactions with the environment. Commonly, an RL problem is formulated as a Markovian Decision Process (MDP) which consists of the elements of state space $\mathcal{S}$, action space $\mathcal{A}$, and reward space $\mathcal{R}$. In detail, at each time step t , the UUV agent observes the current state $s_t \in \mathcal{S}$ and chooses an action $a_t \in \mathcal{A}$. Subsequently, this action will lead to a transition to the successor state s_{t+1} , along with a scalar reward $r_t \in \mathcal{R}$ received from the environment, which quantifies the UUV's performance. The ultimate objective of RL is to discover an effective control policy $\pi(s)$ which maximizes the cumulative long-term return $R_{t_k} = \sum_{t=t_k}^{\infty} \gamma^{t-t_k} r_t$, where $\gamma \in (0, 1]$ is the discount factor, indicating the significance of future rewards, and t_k denotes the initial time step. We present the specific design of the state space, action space, and reward function in the following parts.

A. States space design

The state s_t is a signal that represents environmental information perceived by the UUV agent. In the present context, our aim with the designed RL-based controller is to narrow the error between the UUV and the target path, as well as constantly track the desired heading angles provided by the guidance algorithm. To achieve this, we employ the Line-Of-Sight (LOS) guidance law [37] to calculate the proper directions for the vehicle to follow in the horizontal plane. As illustrated in Fig. 2, e_d denotes the vehicle's current cross-error, ψ_d represents the desired yaw angle, $e_\delta = \psi - \psi_d$ is the direction error to be minimized, and L is the looking ahead distance. The current target path to be followed is represented by $P_{k-1}P_k$, and its slope is symbolized by γ_p . According to the geometry analysis, the desired heading yaw angle can then be given as $\psi_d = \gamma_p + \arctan(-e_d/L)$. In addition, the guidance strategy for the path-following control in the vertical plane is shown in Fig. 3, where e_z denotes the depth error between the current depth and the depth of the P_{LOS} calculated from the vertical plane.

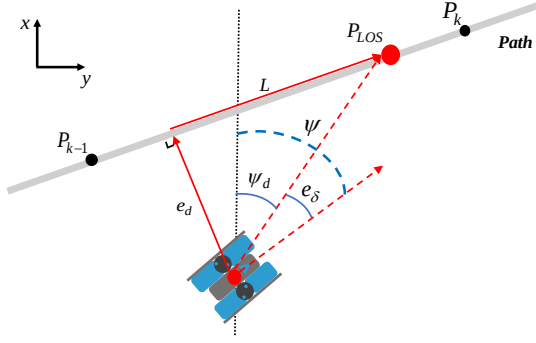


Fig. 2. Illustration of path-following in the horizontal plane.

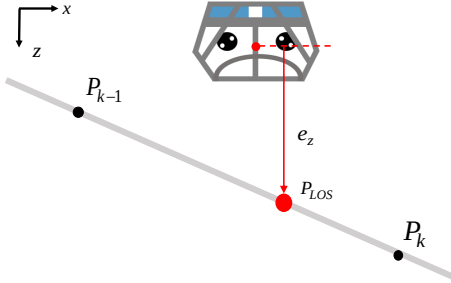


Fig. 3. Illustration of path-following in the vertical plane.

In this way, the current state is defined as

$$\mathcal{S} = \{e_l, e_d, e_z, u, v, w, e_\delta, \sin(\psi_d), \cos(\psi_d), \sin(\psi), \cos(\psi)\}, \quad (2)$$

where $e_l = L$. It should be noted that in the state formulation, the yaw angle ψ and ψ_d are replaced with $\sin(\psi_d), \cos(\psi_d), \sin(\psi), \cos(\psi)$. The advantage of doing so is that ψ is a periodic variable, and the actual direction represented by the yaw angle ψ in the coordinates will remain unchanged if ψ is added with $2k\pi$, i.e., $\sin \psi = \sin(\psi + 2k\pi)$. By using the trigonometric formulation, the ψ -related angle is limited to $(-1, 1)$ from $(-\infty, \infty)$, which largely simplifies the complexity of the RL training process and accelerates the speed of convergence.

B. Action space design

Since the UUV is directly powered by thrusters, we define the thrusters' channel control signal commands as the action space,

$$\mathcal{A} = \{a_1, a_2, a_3, a_4\}. \quad (3)$$

where a_1, a_2, a_3, a_4 symbolizes the control signal commands of x, y, z, yaw channel, respectively.

C. Reward function design

In the RL training process, the reward function plays a crucial role in effectively guiding the agent to meet task requirements. To address the issue of sparse reward, we design a reward function that combines the following components for the path-following control problem.

1) Distance error: To decrease the UUV's error from the target path, a distance reward is established based on the relative distance between the current and target positions. The reward is expressed as:

$$r_e = k_1 \cdot \exp(-k_e \sqrt{e_d^2 + e_l^2 + e_z^2}) - c. \quad (4)$$

Here, k_1, k_e are the weights assigned to penalize the distance component. In some scenarios, the UUV may not take any actions to avoid punishment, causing a deadlock during training. To overcome this issue, a constant penalty c is introduced in this reward function to motivate the UUV to accomplish the task as rapidly as possible.

2) Orientation error: Since the objective of the UUV path-following control is to eliminate the heading error, the orientation function is defined as follows,

$$r_\psi = \begin{cases} k_2 \cdot e^{-k_\psi |e_\psi|}, & \text{if } |e_\psi| < \pi/2 \\ k_2 \cdot -e^{k_\psi (|e_\psi| - \pi)}, & \text{if } |e_\psi| \geq \pi/2 \end{cases} \quad (5)$$

where k_2, k_ψ are the weights given to penalize the direction error. As the formulation indicates, a higher reward will be received if the tracking error and e_ψ are zero or near zero, while a lower reward will be returned if the value diverges from zero.

However, uncertainties introduced by the environments, such as the time-varying hydrodynamics and external disturbances, pose a significant challenge in the training process. These factors make it difficult for the training process to converge effectively, causing the UUV to frequently deviate from the desired path and struggle to complete the path-following within a single episode. As a result, the UUV may not gain any valuable experience from the training process. In light of this, we define an additional reward r_ρ to further guide the vehicle during training and accelerate the convergence speed.

3) This additional reward r_ρ is defined as:

$$r_\rho = \begin{cases} 0.5, & r_e < \rho_{min} \\ 0, & \rho_{min} \leq r_e < \rho_n \\ -0.5, & \rho_n \leq r_e < \rho_{max} \end{cases} \quad (6)$$

where $r_e = \sqrt{e_d^2 + e_l^2 + e_z^2}$, and ρ_n is a dynamic radius, defined as a range to constrain the UUV. Moreover, ρ_{min} and ρ_{max} are user-defined constants, satisfying $0 < \rho_{min} < \rho_{max}$. Here we set $\rho_{min} = 1m$ and $\rho_{max} = 10m$ based on the considered UUV specifications. Supposing the vehicle is currently at the position of $P(x, y, z)$, and P_o is defined as the pedal of the path with respect to P . **A sphere with a dynamic radius ρ_n centered at P_o is defined**, of which the upper bound is $\rho_{max} = 10m$ and the lower bound is $\rho_{min} = 1m$. During the training process, if UUV stays within the range of ρ_{min} , it will be given the highest reward of 0.5. When it runs out of ρ_n , it will be punished with -0.5. The value of the dynamic radius ρ_n is determined by

$$\rho_n = \begin{cases} 10, & \bar{h}_k < 15\% \\ -5 \ln \bar{h}_k, & 15\% \leq \bar{h}_k < 80\% \\ 1, & \bar{h}_k \geq 80\% \end{cases} \quad (7)$$

where $\bar{h}_k$ is a measure to evaluate the tracking performance over the most recent n_1 ($n_1 < k$) episodes, and can be calculated by the following equation,

$$\bar{h}_k = (h_{k-n_1} + h_{k-n_1+1} + \dots + h_{k-1}) / n, \quad (8)$$

$$h_k = \frac{\sum_{i=1}^{N_k} r_i - r_{min}}{N_k * (r_{max} - r_{min})}. \quad (9)$$

Here, h_k is referred to as the reward ratio of k th episode, decided by the accumulated reward of the current episode and the total defined reward; N_k is the total step contained in the k th episode; r_{min} and r_{max} are the minimal and the maximum rewards of the defined one-step reward, respectively; r_i is the obtained reward in one step.

With all these designs, one can see that as the training performance improves, it becomes increasingly difficult to obtain the extra reward r_ρ . In cases where the performance fails to improve within a certain number of episodes, a penalty will be received.

The whole reward for the path-following control is concluded as $r = r_e + r_\psi + r_\rho$.

IV. PATH-FOLLOWING CONTROL WITH IMPROVED TD3

Based on the RL framework designed in Section III, we introduce an improved TD3 algorithm to address the UUV path-following control problem. Its main structure and dataflow are illustrated in Fig. 4. The focus of Fig. 4 lies on a basic TD3 module that contains six NNs, as well as two essential components that have been enhanced to improve the algorithm's performance: an improved experience replay and the policy smoothness regulation. Specifically, the improved experience replay employs both TD-errors and episodic return to evaluate the significance of each experience, and concurrently sampled experiences based on both sets of significances. The policy smoothness regulation introduces a dynamic parameter, which adaptively adjusts the smoothness constraint based on the UUV's current state, which adaptively adjusts the smoothness constraint based on the UUV's current state. Details of these components are discussed in the following subsections.

A. Algorithm mechanism of TD3

TD3 is a state-of-the-art DRL method used to solve control problems with continuous action space. It is an improvement over the DDPG algorithm, offering better training stability and mitigating the limitations of DDPG, including overestimation, overfitting, and sensitivity to hyperparameters. Similar to DDPG, TD3 adopts an actor-critic structure, in which the critic is used to evaluate an action-state value function $Q_\pi(s, a)$. Specifically, at the timestep t , $Q_\pi(s_t, a_t)$ represents the long-time reward when the action a_t is taken at s_t and follows the control policy $\pi(s)$ thereafter. In such a manner, the Q -function essentially satisfies,

$$Q_\pi(s_t, a_t) = r_t + \gamma Q_\pi(s_{t+1}, \pi(s_{t+1})). \quad (10)$$

The ultimate goal of RL is to learn an optimal control policy $\pi^*(s)$ that maximizes $Q_\pi(s_t, a_t)$, formalized by

$$\pi^*(s) = \arg \max_{\pi} Q_\pi(s_t, a_t). \quad (11)$$

In general, neural networks are utilized in the actor-critic structure to solve Eq. (10) and Eq. (11). As shown in Fig. 4, TD3 is mainly constructed by six neural networks. Particularly, two evaluate critics are used to estimate the Q -functions, namely Q_{θ_1} with parameter θ_1 , and Q_{θ_2} with parameter θ_2 . In addition, an evaluate actor Q_π with parameter ϕ is used for policy updating. Each actor and critic is accompanied by a target network to guarantee a stable training process, denoted by $Q_{\theta'_1}$, $Q_{\theta'_2}$, and $Q_{\pi'}$. The updating of the evaluate critics are driven by the following loss function,

$$L(\theta_j) = \frac{1}{N} \sum_{i=1}^N w_i \delta_i^2, \quad (12)$$

$$\delta_i = r_i + \gamma \min_{j=1,2} Q_{\theta'_j}(s_{i+1}, \pi_{\phi'}(s_{i+1}) + \epsilon) - Q_{\theta_j}(s_i, \pi_{\phi'}(s_i)) \quad (13)$$

where N is the batch size of the selected transitions at each step, δ_i is known as the time differencing error (TD-error), and w_i is the importance sampling weight of i^{th} sample. As Eq. (13) indicates, the minimal estimated Q -value is selected for calculating the target value estimation, so as to reduce the overestimation bias problem in traditional Q -learning. In addition, to ensure a more smoothly updating process for critics, and avoid the problem of over-fitting, a noise signal is added to the target actor as a regularization term, $\epsilon \sim \text{clip}(\mathcal{N}(0, \sigma), -\bar{c}, \bar{c})$. Here, σ is the noise standard deviation, $\mathcal{N}$ represents the standard normal distribution, and $\bar{c}$ is the noise limits.

The evaluate actor is updated by maximizing the learned Q -function, and the deterministic policy gradient strategy is used here,

$$\nabla_{\phi} J(\phi) = \frac{1}{N} \sum_{i=1}^N \nabla_a Q_{\theta}(s_i, a_i)_{a=\pi_{\phi}(s_i)} \cdot \nabla_{\phi} \pi_{\phi}(s_i), \quad (14)$$

where $J(\phi) = \frac{1}{N} \sum_{i=1}^N Q_{\theta}(s_i, a_i)_{a=\pi_{\phi}(s_i)}$ is the expected return to be maximized, $\nabla_a Q_{\theta}(s_i, a_i)_{a=\pi_{\phi}(s_i)}$ is the gradient of Q -value with respect to action, and $\nabla_{\phi} \pi_{\phi}(s_i)$ is the gradient with respect to the evaluate actor parameter ϕ . Furthermore, to provide a stabilized training process, the parameters of the target networks are updated to track the evaluated networks by following the soft replacement law,

$$\begin{aligned} \theta' &\leftarrow (1 - \bar{\tau})\theta' + \bar{\tau}\theta, \\ \phi' &\leftarrow (1 - \bar{\tau})\phi' + \bar{\tau}\phi \end{aligned} \quad (15)$$

where $\bar{\tau} \in (0, 1]$ is the soft factor, determining the updating rate of the target networks. It is noteworthy that the updating frequency in actor and critic is inconsistent in TD3. Generally, the critic network is updated at a higher frequency than the actor network, so as to first minimize the Q -value error before introducing the policy updating.

B. Policy smoothness method

One challenge in this RL-based path-following control problem comes from the control signal oscillation. The intricate structure of deep neural networks employed in RL introduces a sensitivity where even a minor change in input can yield noteworthy differences in output, resulting in the obtained

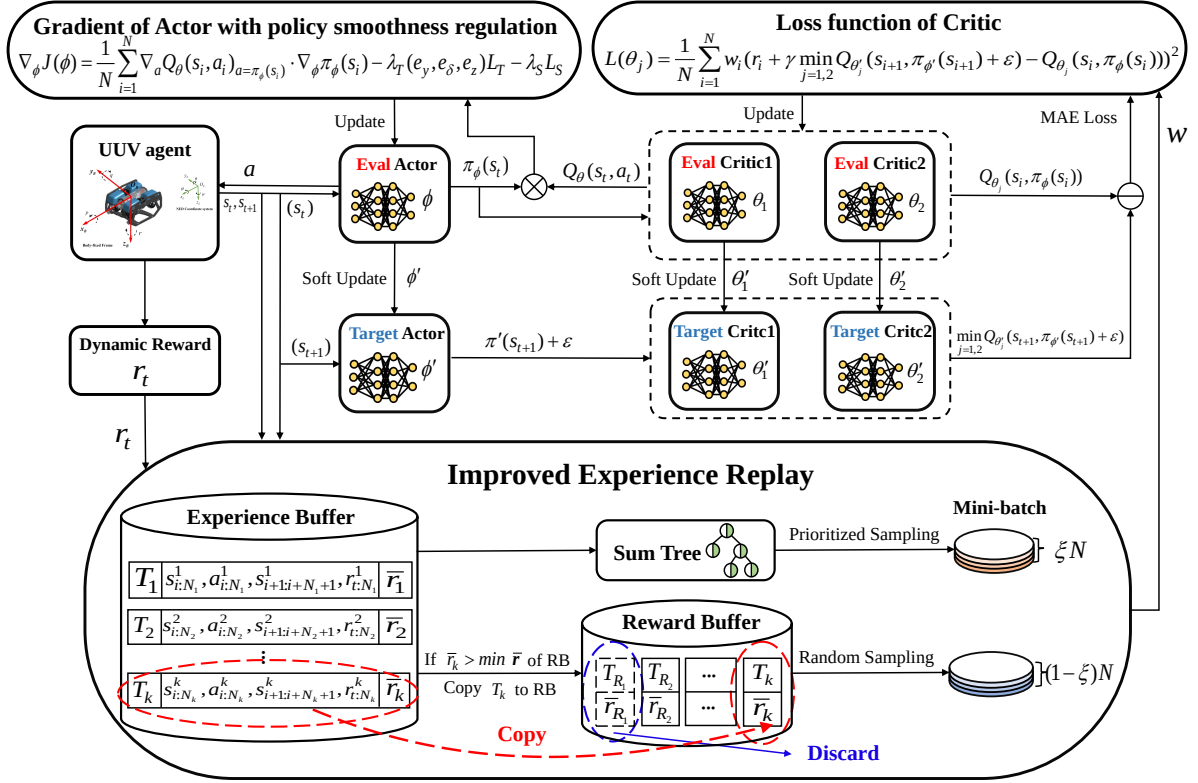


Fig. 4. Path-following control structure of the UUV based on TD3-IMP.

control policy being extremely non-smooth [41, 42]. Such issues can lead to excessive system energy consumption and severe mechanical wear on actuators, posing a significant drawback in practical applications. To address this, Mysore [35] originally introduced a method called Conditioning for Action Policy Smoothness (CAPS). A notable advantage of this approach is that it directly ensures the smoothness of the control policy within the RL learning process, eliminating the need for adjustments during the implementation stages. In detail, a temporal smoothness regularization term, L_T , and a spatial smoothness term, L_S , are integrated into the actor loss,

$$\begin{aligned} J^{new}(\phi) &= J(\phi) - \lambda_T L_T - \lambda_S L_S, \\ L_T &= \|\pi(s_t) - \pi(s_{t+1})\|_2, \\ L_S &= \|\pi(s_t) - \pi(\tilde{s}_t)\|_2 \end{aligned} \quad (16)$$

where λ_T and λ_S are the gains of each regularization and $\tilde{s}_t$ is sampled from a normal distribution centered at s_t , following $\mathcal{N}(s_t, \sigma)$. Therein, the term $\lambda_T L_T$ is used to suppress the temporal variation of the policy, implying that the change of actions between successive states should be minimized. And $\lambda_S L_S$ is used to achieve policy local smoothness, encouraging the agent to select similar actions when the state is perturbed.

To effectively apply this method to the UUV path-following control, the value of the smoothness parameter λ_T should be determined carefully. A small value may render the term ineffective, making the desired smoothing effect hard to achieve. While a large value will excessively smooth the policy, and the agents may lose the ability to adapt to scenarios demanding

action adjustments, especially when UUV is at a curve and requires quick steering changes. This is exemplified in Fig. 5, where the predefined path to be followed is represented by the black line. The blue dots represent the trajectory series $\{P_t, P_{t+1}^1, \dots, P_{t+6}^1\}$ generated under a CAPS of λ_{T_1} , while the red dots represent the trajectory series $\{P_t, P_{t+1}^2, \dots, P_{t+6}^2\}$ generated under a CAPS of λ_{T_2} , with $\lambda_{T_1} > \lambda_{T_2}$. The control policies under these two conditions are denoted as π_1 and π_2 , with the corresponding control inputs shown in the lower subfigure, where π_2 clearly exhibits more oscillations than π_1 . We segment the example in Fig. 5 into two situations, as shown in it. For Situation I, though the control signal of π_1 is more smoothed than π_2 , the tracking error under π_1 is noticeably larger than that of π_2 . While for Situation II, although the tracking error of π_2 is comparable with π_1 , it exhibits larger oscillations than π_1 .

This phenomenon indicates that the policy with λ_{T_1} has been over-smoothed, leading to undesired tracking performance. However, the policy with the smaller value λ_{T_2} has limited ability to constrain the control oscillation, despite it achieves smaller tracking errors. These results imply that the strength of the smoothness should be dynamically adjusted according to the UUV's states rather than being strictly fixed, as in CAPS. Here, we propose a modified policy smoothness method to this end, where the strength of the smoothness is decided based on the tracking error of the corresponding state. The relation is formulated as $\lambda_T(e_d, e_\delta, e_z) = k_{D1} \exp(-k_{D2}(|e_d| + |e_\delta| + |e_z|))$, where

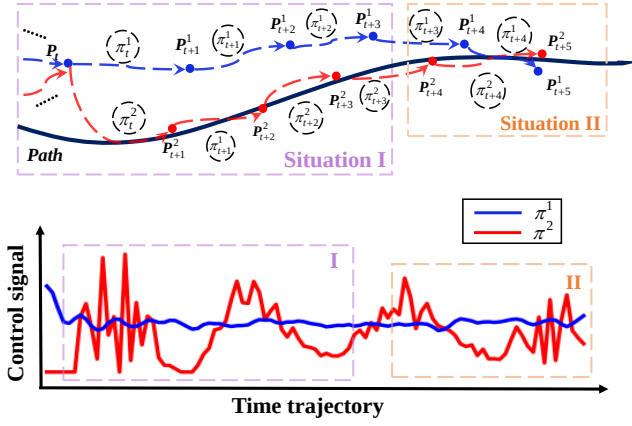


Fig. 5. Illustration of the CAPS strategy under the smoothness gains of λ_{T_1} and λ_{T_2} .

λ_T is decided by the cross-track error e_d , the direction error e_δ , and the depth error e_z . If the tracking error exceeds a proper range, the strength of smoothness λ_T is decreased to weaken the regularization effect, such that the policy is trained to prioritize the elimination of the tracking error over the realization of policy smoothness. Conversely, if the tracking error is maintained within a reasonable range, a larger λ_T is applied to ensure a smooth action trajectory. By employing this method, the criteria of Eq. (16) is now formulated as

$$\begin{aligned} J^{new}(\phi) &= J(\phi) - \lambda_T(e_d, e_\delta, e_z)L_T - \lambda_S L_S, \\ \lambda_T(e_d, e_\delta, e_z) &= k_{D1} \exp(-k_{D2}(|e_d| + |e_\delta| + |e_z|)), \\ L_T &= \|\pi(s_t) - \pi(s_{t+1})\|_2, L_S = \|\pi(s_t) - \pi(\tilde{s}_t)\|_2. \end{aligned} \quad (17)$$

And the actor-network updating rules Eq.(14) in the previous section are now based on Eq. (17). Note that in our case the spacial parameter λ_S remains fixed, since the random noise value is usually relatively small compared to the original state. In this study, the sensor noise is defined following the Gaussian distribution, which is described in [43].

C. Improved experience replay buffer

During the training process, the UUV transitions are stored in an experience replay buffer in the form of (s_t, r_t, s_{t+1}, a_t) , and will be uniformly sampled to update the evaluate critic networks. However, this mechanism has low training efficiency as the learning value of each transition cannot be well demonstrated. To address this, prioritized experience replay (PER) is proposed by Schaul et al.[44]. This method leverages temporal difference (TD) error as a crucial metric, reflecting the importance and novelty of a transition in relation to the agent's current expectations. By assigning higher probabilities to transitions with elevated TD-errors, PER promotes the agent's learning from previously unencountered states. This, in turn, bolsters the agent's exploratory capabilities.

In the context of UUV path-following problem, the tracking performance is reflected by the episode return, where a high-accuracy path-following performance in one episode will receive a high episode return. More specifically, higher episode returns are composed of high-reward actions, which

enable the agent to navigate through the desired path with smaller tracking errors. Therefore, prioritizing the sampling of transitions from higher episodic returns is of significance, since learning from these experiences enables the agent to make rational decisions to reach the target. In this regard, we propose an improved experience replay sampling mechanism to accelerate the training convergence speed, where the TD-error and the averaged episode return are both considered as sampling indicators.

In detail, two memory buffers are constructed, namely a normal memory buffer $\mathcal{M}_T$ and a reward buffer $\mathcal{M}_R$, as shown in Fig. 4. Particularly, the mini-batch $\mathcal{B}$ used for network training is constructed by $\mathcal{M}_T$ and $\mathcal{M}_R$, with a user-defined ratio ξ employed to determine the sampling portion of each buffer. For $\mathcal{M}_T$, its function is similar to the memory buffer in the vanilla TD3, where all the transitions are stored sequentially. Here, PER is adopted as the sampling method for $\mathcal{M}_T$, where the priority is based on the absolute value of TD errors $|\delta|$. In $\mathcal{M}_T$, the sampling priority of i^{th} transition is defined as $p_i = |\delta_i| + \epsilon_1$, where ϵ_1 is a small positive value to prevent zero sampling probability. While for $\mathcal{M}_R$, transitions from high episode reward trajectory are stored based on their averaged episode return $\bar{r}$, which is given by $\bar{r} = \sum_{i=1}^{N_k} r_i / N_k$. Specifically, if the average episode return of the current trajectory is higher than one of the previous trajectories stored in $\mathcal{M}_R$, whole transitions from the current trajectory will be directly copied to $\mathcal{M}_R$, and the previous trajectory with the lowest average reward will be abandoned. Given that these experiences already yield high rewards and exhibit minimal variability, they can be considered of equal importance. Therefore, we employ uniform random sampling in $\mathcal{M}_R$ to reduce computational complexity.

Since the prioritized sampling mechanism changed the original sampling distribution, the importance sampling weights (ISW) needs to be applied to correct the gradient of the replayed transitions. For transitions sampled from different replay buffers, their corresponding ISWs are adjusted according to their buffer size and calculated as follows,

$$\begin{aligned} w_{j, \mathcal{M}_R} &= \frac{1}{m_r}, \\ w_{q, \mathcal{M}_T} &= \frac{(m_t \cdot P(q))^{-\beta}}{\max_i \{w_{i, \mathcal{M}_T}\}}, \end{aligned} \quad (18)$$

where $j = 1, 2, \dots, (1 - \xi)N$ is the j^{th} transition sampled from $\mathcal{M}_R$; $q = 1, 2, \dots, \xi N$ is the q^{th} transition sampled from $\mathcal{M}_T$; β is a coefficient, deciding to what level the gradient gets corrected from the replayed transition; $P(q) = p_q / (\sum_{i=1}^m p_i)$ is the probability of the transition be to sampled, m_r and m_t are the size of $\mathcal{M}_R$ and $\mathcal{M}_T$, respectively. Particularly, if a transition is both sampled from $\mathcal{M}_R$ and $\mathcal{M}_T$, its ISW is given as $w_i = w_{i, \mathcal{M}_R} + w_{i, \mathcal{M}_T}$, which is a linear combination of the weights in (18). The pseudocode of the improved experience replay is presented in Algorithm 1.

D. The whole training process

The complete implementation process of our TD3-IMP is presented by taking new designs described in previous

Algorithm 1 Improved Experience Replay Mechanism

```

1:  $\mathcal{M}_T$ : the memory buffer, with size  $m_t$ .
2:  $\mathcal{M}_R$ : the reward memory buffer, with size  $m_r$ .
3:  $\mathcal{B}$ : mini batch in each sampling, with size  $N$ .
4:  $\bar{r}_k$ : the averaged reward for one episode.
5:  $\bar{r}_{\min}$ : the minimum averaged reward stored in  $\mathcal{M}_R$ .
6: for each episode do
7:   for each sampling do
8:     Sample  $[\xi N]$  transitions  $[s_t, a_t, s_{t+1}, r_t]$  from  $\mathcal{M}_T$ .
9:     Calculate ISW for every sampled transition, with  $w_{q, \mathcal{M}_T} = [m_t \cdot P(q)]^{-\beta} / \max_i \{w_{i, \mathcal{M}_T}\}$ .
10:    Sample  $[(1 - \xi)N]$  transitions  $[s_t, a_t, s_{t+1}, r_t]$  from  $\mathcal{M}_R$ .
11:    Calculate ISW for every sampled transition, with  $w_{j, \mathcal{M}_R} = 1/m_r$ .
12:    Group all sampled transitions into a mini-batch  $\mathcal{B}$ .
13:   end for
14:   if  $\bar{r}_k \geq \bar{r}_{\min}$  then
15:     Copy the whole episode of the transitions into  $\mathcal{M}_R$ , and remove the episode of transitions with  $\bar{r}_{\min}$ .
16:   end if
17: end for

```

sections and is summarized in Algorithm 2. Its main process is described as follows. At the beginning of each training step, the state of the UUV system, calculated by Eq. (2), serves as the input to the TD3-IMP structure. Subsequently, the evaluate actor network produces an action based on this state, and the immediate reward for that action within the current state is determined using the designed reward function. These experience transitions are then stored in the replay buffers $\mathcal{M}_T$ and $\mathcal{M}_R$. Once sufficient experience has been collected in both $\mathcal{M}_T$ and $\mathcal{M}_R$, the agent training is initiated. During the training phase, a minibatch $\mathcal{B}$ of size N is sampled via the procedures detailed in Algorithm 1. The evaluate twin critic network is then updated using the critic updating law in Eq. (12), (13). Additionally, the actor is updated every d iteration, following the modified smoothness policy given by Eq. (17). Then the target critic and actor networks are updated through the soft replacement method described in Eq. (15). This iterative training process continues until it reaches the maximum episode number.

V. SIMULATION, EXPERIMENTS, RESULTS, AND ANALYSIS

In this section, we present and discuss the simulation results of the trained control policy for the underwater vehicle path-following tasks. We first train the vehicle to follow a helix path with our proposed TD3-IMP method and compare its performance with other advanced reinforcement learning methods, including DDPG, vanilla TD3, and TD3 with prioritized experience replay (TD3-PER). Meanwhile, to demonstrate the effectiveness of our designed control policy smoothness method, simulations in a three-dimensional path scenario are conducted. Subsequently, the trained policy network is applied to more complicated paths following scenarios to test its generalization ability, which involves two-dimensional horizontal paths and a three-dimensional underwater path. It is expected that the learned control strategy in a relatively simple scenario can be adapted to complete new path-following tasks without further training.

Our simulations were performed on Ubuntu 18.04, and the control strategy was established by Pytorch 1.7. In our design, both the actor and the critic network have two hidden

Algorithm 2 TD3-IMP for the UUV path-following control

```

1: Initialize the evaluate critic networks  $\theta_1, \theta_2$ , evaluate actor network  $\phi$  with random parameters.
2: Initialize the target networks with  $\theta'_1 = \theta_1, \theta'_2 = \theta_2, \theta' = \phi$ .
3: Initial the size of the replay buffer  $m_t, m_r, N$ .
4: Initialize user-defined parameters involved in the algorithm,  $k_e, k_\psi, c, \xi$ .
5: for each episode do
6:   Observe the initial state of the UUV  $s_0$ .
7:   for  $t = 0$  to step do
8:     Choose the action  $a_t$ .
9:     Apply  $a_t$  to the UUV dynamics, calculate the reward  $r_t$  from the environment, and observe  $s_{t+1}$ .
10:    Store the transition  $(s_t, a_t, s_{t+1}, r_t)$  in  $\mathcal{M}_T$ .
11:    if Training = True then
12:      Sample mini-batch of  $N$  transitions to build  $\mathcal{B}$  based on Algorithm 1
13:      Update the parameter  $\theta_1, \theta_2$  of the evaluate critic networks by (12), (13).
14:    end if
15:    if  $t \bmod d$  then
16:      Update the parameter  $\pi$  of the evaluate actor network by the modified smoothness policy in (17).
17:      Update the target networks  $\theta'_1, \theta'_2, \pi'$  with the soft replacement method by (15).
18:    end if
19:    end for
20:    Store the episode of the transitions into  $\mathcal{M}_R$ , if  $\bar{r}_k \geq \bar{r}_{\min}$ .
21: end for

```

layers, where the number of neurons in each hidden layer is 256 respectively. Considering the limited power of the UUV thrusters, the output action should be maintained in a given range. Here, we adopt tanh units as the activation functions of the actor output layer, such that the action space can be scaled to $[-1, 1]$. The UUV simulator is designed according to the dynamic modeling given in Section II. To ensure that the learned policy is universally applicable in any working environment, we randomly selected hydrodynamic parameters within a 20% uncertainty range during the training process. In addition, to enhance the model's ability to resist external disturbances, the influence of water waves is introduced to the environment which takes the form of the sinusoid and is supposed as $\tau_{\Delta x} = 6 * rand(-1, 1) \sin(0.1t)N$, $\tau_{\Delta y} = 5 * rand(-1, 1) \cos(0.2t)N$, $\tau_{\Delta z} = 4 * rand(-1, 1) \cos(0.3t)N$ [45]. The hyperparameters involved in the training are summarized in Table I.

TABLE I
PARAMETERS FOR THE SIMULATION TRAINING.

Parameter symbol	Value
Soft replacement rate, $\bar{\tau}$	0.01
Discount factor, γ	0.99
Learning rate of actor and critic, l_a, l_c	0.0003, 0.001
Delayed policy update iteration	2
Size of $\mathcal{M}_R, \mathcal{M}_T$	200000, 40000
Size of $\mathcal{B}$	128
Initial and maximal values of ξ	0, 0.5
Incremental values of ξ	0.00125
Initial and maximal values of β	0.4, 1
Incremental values of β	0.001
Smooth parameter k_{D1}, k_{D2}	0.022, 1
Reward parameters $k_1, k_2, k_e, k_\psi, c, n_1$	3, 5, 7, 4, 0.3, 5

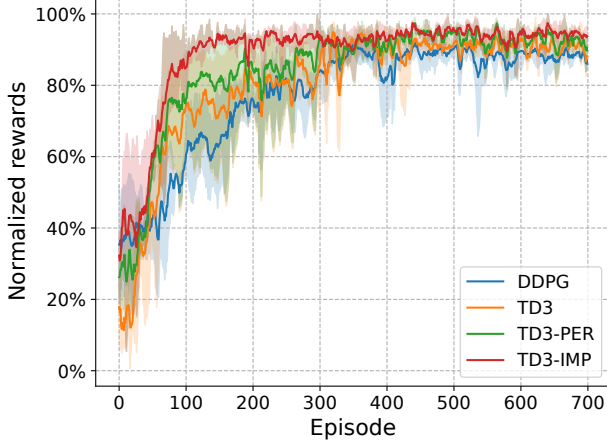


Fig. 6. Normalized average rewards under the agent of DDPG, TD3, TD3-PER, and TD3-IMP.

A. Training results

In the training stage, the reference trajectory is defined as $x_r = 2 - 2\cos(\varsigma)m$, $y_r = 2\sin(\varsigma)$, $z_r = -2\varsigma - 0.5$, where $\varsigma \in [0, 2\pi]$. The initial position and the heading angle of the UUV are randomly selected to produce a variety of encounter conditions with the target path. To be specific, the position is initialized with $x_0 = x_{r0} + \text{rand}(-0.5, 0.5)$ m, $y_0 = y_{r0} + \text{rand}(-0.5, 0.5)$ m, $z_0 = z_{r0} + \text{rand}(-1, 0)$ m, $\psi_0 = (-\pi, \pi)$. The reinforcement learning controller is trained with 700 episodes during the learning, and for each episode, the training step is limited to 400 time steps. If the UUV's lateral error deviates 10 m away from the target path, which implies a failure to complete the tracking task or the maximum time step is reached, the current episode will be terminated. Then a new episode will begin with another starting position.

The changing trend of the accumulated rewards received from four different agents under four different DRL methods is depicted in Fig 6, where the solid line is the average reward under five trials, and the shaded region represents their corresponding standard deviation. The rewards are normalized by the maximum rewards defined during the learning process of all algorithms. As Fig. 6 illustrates, the accumulated reward of all four agents increases as the training proceeds and tends to be stabilized after 600 episodes. Among the four methods, TD3-IMP achieves the best performance and fastest convergence speed, starting to converge at around 150 episodes and quickly maintaining rewards within 95% of the highest reward. In comparison, TD3-PER, TD3, and DDPG agents take approximately 300, 400, and 450 episodes, respectively, to reach a comparable level.

To better understand the behavior of the control policy across various training phases, we selected key stages from the training of four algorithms and extracted the corresponding policy parameters of their actor networks. These parameters were subsequently employed as the control system for the UUV's path-following mechanism. For comparison, the UUV is consistently initialized at the same starting position $(x_0, y_0, z_0, \psi_0) = (0, -0.1, 0, 0)$ across all episodes. The

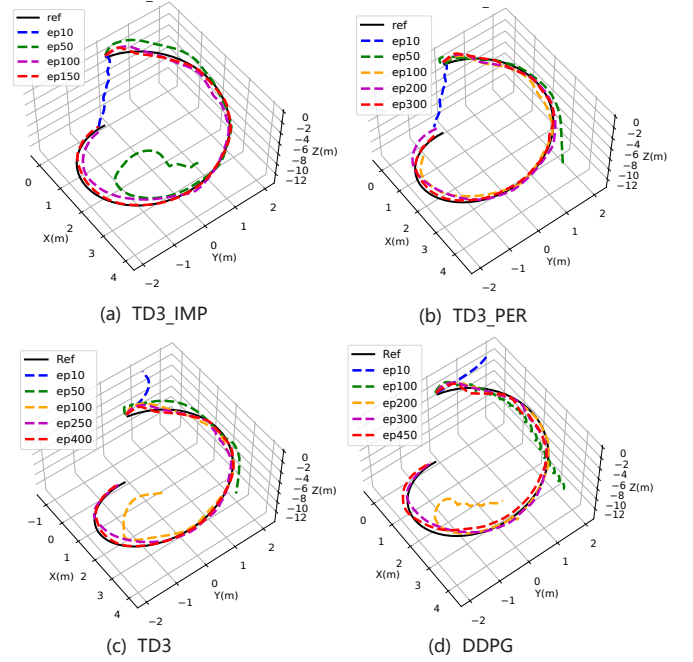


Fig. 7. Trajectories of the UUV helix path-following control with the agent (a)TD3-IMP, (b)TD3-PER, (c)TD3, (d)DDPG.

learning trajectories of the UUV's path-following process, as directed by DDPG, TD3, TD3-PER, and TD3-IMP, are illustrated in Fig. 7. Additionally, Fig. 8(a) and Fig. 8(b) present the corresponding average tracking error and yaw error, respectively. Notably, by the 10th episode of the training stage, the UUV quickly fails with all four agents. This can be attributed to the fact that, in the early stages of training, the UUV struggles to accumulate beneficial experiences, thereby leading to deficient path-following capabilities. As the training progresses, the agents of TD3-IMP and TD3-PER demonstrate improved path-following ability by the 100th episode. In particular, the performance of the TD3-IMP agent surpasses that of the TD3-PER, exhibiting a smaller tracking error in both distance and yaw, as evident in Fig. 8(a) and Fig. 8(b). After 150 episodes of training, the TD3-IMP agent is capable of completing the entire path with satisfactory results. In contrast, the agents of TD3-PER, TD3, and DDPG require additional training episodes. Specifically, the TD3-PER agent requires an extra 150 episodes to minimize the depth error, ultimately converging at the 300th episode. Meanwhile, the agents of TD3 and DDPG are unable to complete the task until the 250th and 300th episodes, respectively. To achieve a similar performance with the TD3-IMP agent at the 100th episode, they need an additional 250 and 300 episodes, respectively.

From Fig. 8, it can be observed that the tracking error and the yaw error of TD3-IMP and TD3-PER agents fall quicker than the other agents and the tracking error of TD3-IMP is smaller than that of TD3-PER, which is consistent with the learning curve in Fig. 6. These findings validate that our method can effectively accelerate the convergence rate of the training process while also ensuring the attainment of high

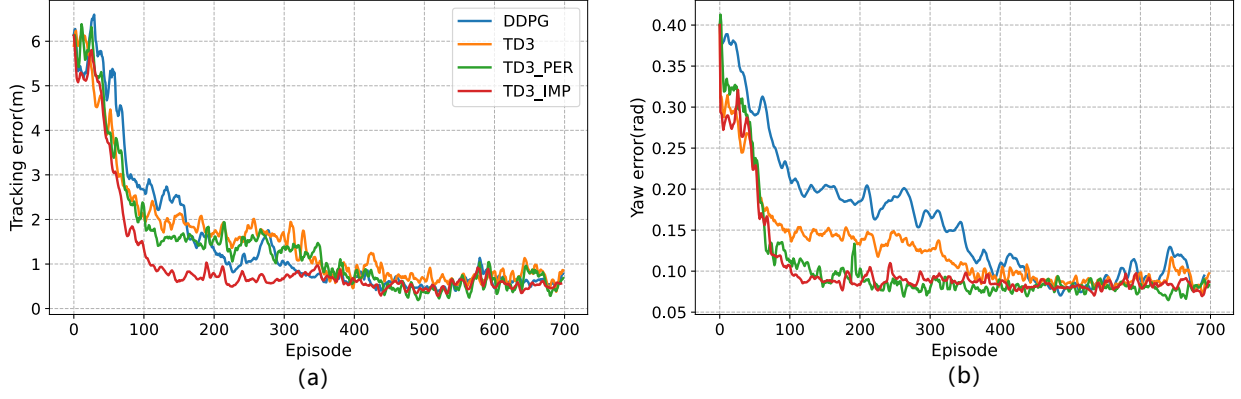


Fig. 8. The averaged tracking error of different learning agents under different training episodes. (a)Tracking error (b)Yaw error.

training rewards. It provides superior tracking performance when compared to relevant RL techniques.

B. Control policy smoothness

To further evaluate the effectiveness of our proposed policy smoothness strategy, simulations are conducted by comparing the performance of three agents trained under different smoothness parameters: $\lambda_T = 0.005$, $\lambda_T = 0.02$ and a dynamic λ_T (our method). The training environment and conditions are identical to the previous subsection with only the smooth parameters being different. The learning curves of these three agents are shown in Fig. 9. It can be observed that the agents trained with $\lambda_T = 0.005$ and the dynamic λ_T achieve the highest rewards while the agent with $\lambda_T = 0.02$ has the worst performance. We then apply the trained policies to track a curve path, where the waypoints are set as $[0, 0, -1]$, $[6, 8, -4]$, $[-1, 14, -6]$, $[2, 25, -10]$, $[35, 0, -13.5]$. To avoid unrealistic sharp turns, we used a quadratic polynomial interpolation method to smooth the path.

The tracking results of these three agents are shown in Fig. 10, Fig. 11, and Table II. Here, the channel signal commands are converted into each thruster's signal commands through the thrust configuration matrix H , to provide a clear illustration. The performance of each agent is evaluated in terms of tracking error and the smoothness evaluation matrix introduced in [35], $Sm = 2/nf_s \sum_{i=1}^n M_i f_i$. Here, M_i is the amplitude of the i th frequency component f_i , and f_s is the sampling frequency. It can be found from Fig. 11(a)-(c) that although the control signal of the agent trained with $\lambda_T = 0.02$ shows the smallest S_m of only 1.01×10^{-4} while tracking down the curve path, its tracking error is larger than the other two agents. Furthermore, this agent fails to complete sharp turnings, which is consistent with the low learning reward as depicted in Fig. 9. The reason for this lies in the inappropriate excessive smoothness of the control policy, as it prioritizes achieving smaller fluctuations in the thruster signal but at the cost of compromising tracking accuracy. In other words, the over-smoothness of the policy limits the UAV's ability to adapt to situations that require action changes, such as making a turning, ultimately leading to task failure.

For the agent trained with $\lambda_T = 0.005$, while it can successfully achieve the target path with a comparatively small

tracking error, the thruster signals exhibit a high degree of variability and aggressiveness as demonstrated in Fig. 11(d), which will cause system tear-and-wear and is not desirable in real applications. On the other hand, the agent trained with the dynamic λ_T achieves the most desirable performance. Though the tracking error in e_{xy} and e_z is slightly larger than that of $\lambda_T = 0.005$, its action smoothness shows a significant improvement, where the value of S_m reduces to 2.09×10^{-4} . These results validate that the proposed dynamic policy smoothness method can effectively achieve a balance between tracking accuracy and control signal smoothness, which provides a more suitable option for real-world applications.

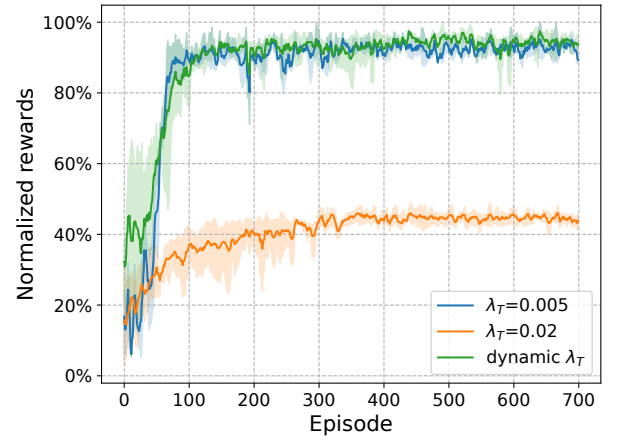


Fig. 9. Normalized average rewards under the agent trained with $\lambda_T = 0.005$, $\lambda_T = 0.02$ and dynamic λ_T .

TABLE II
THE MEAN ABSOLUTE ERROR AND THE SMOOTHNESS MATRIX VALUE OF 3D PATH-FOLLOWING UNDER DIFFERENT λ_T .

λ_T	$e_{xy}(m)$	$e_z(m)$	$e_\delta(rad)$	Sm(%)
0.005	0.178	0.111	0.095	1.2550
0.02	0.734	2.965	0.324	0.0101
dynamic	0.211	0.158	0.092	0.0209

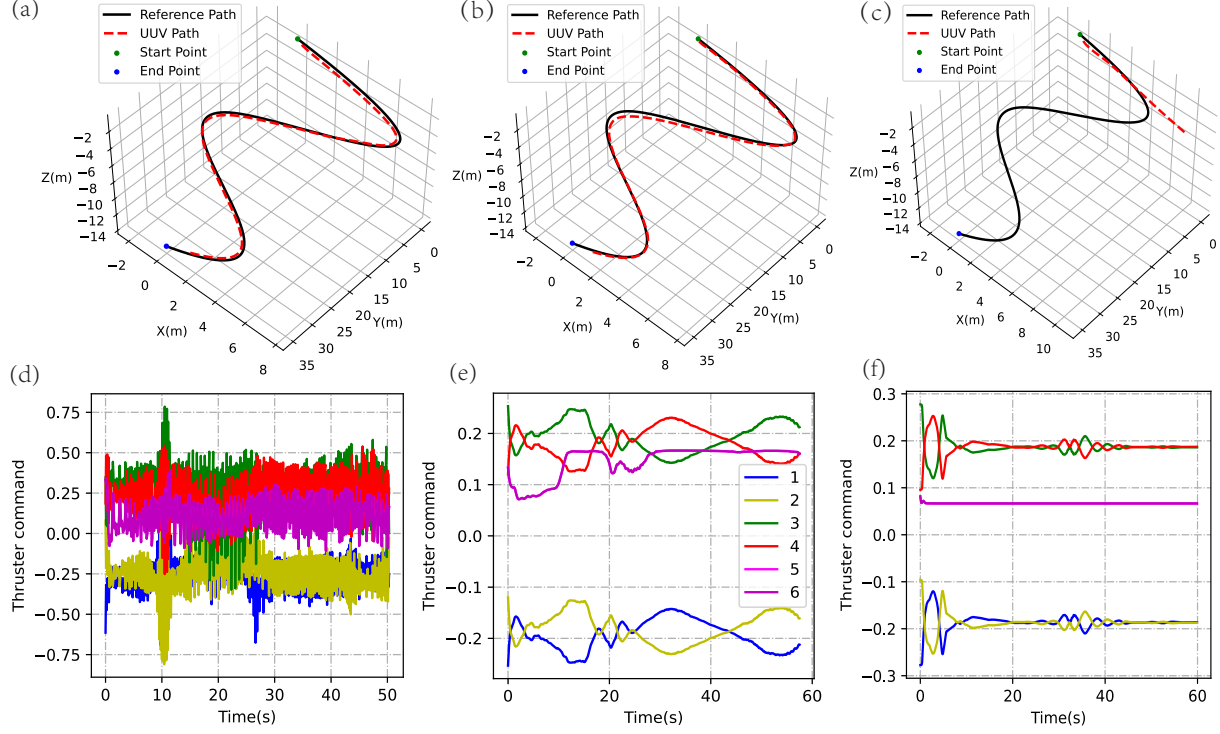


Fig. 10. Trajectory and thrusters commands of UUV with different smooth methods. (a)-(c). Trajectory trained under $\lambda_T = 0.005$, dynamic λ_T , $\lambda_T = 0.02$. (d)-(f). Thruster commands trained under $\lambda_T = 0.005$, dynamic λ_T , $\lambda_T = 0.02$.

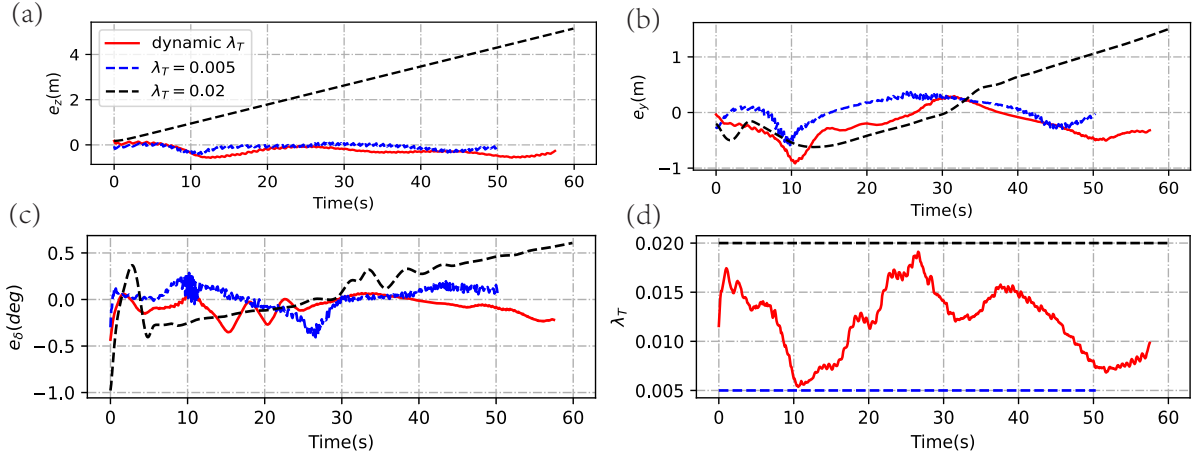


Fig. 11. Simulation results of the agent trained under $\lambda_T = 0.005$, dynamic λ_T , $\lambda_T = 0.02$. (a) Position error in the horizontal plane. (b) Position error in the vertical plane. (c) Yaw error. (d) Change of the value of λ_T .

C. Generalization to complicated path scenario

To further validate the generalization ability of the proposed TD3-IMP algorithm, the final control policy trained with the proposed TD3-IMP in the previous subsection is saved. The UUV is deployed with these parameters and then tested in different paths following scenarios, where the desired paths can be either two-dimension or three-dimension. Here, the widely-used PID controller is also employed to make the comparison analysis with the proposed controller.

1) *2D path tracking*: The TD3-IMP controller is first applied to a 2D path-following scenario. Since the 2D scenario only contains the horizontal path-following control, the depth-

TABLE III
THE MEAN ABSOLUTE ERROR VALUE OF THE PID CONTROLLER AND THE TD3-IMP CONTROLLER UNDER DIFFERENT PATHS.

Path type	PID	TD3-IMP
sine wave(m)	0.015	0.012
comb path(m)	0.029	0.018
closed curve(m)	0.183	0.025
eight-shaped path(m)	0.593	0.029

related input state of the TD3-IMP controller is set to zero. Four different shapes of 2D paths are set as reference paths,

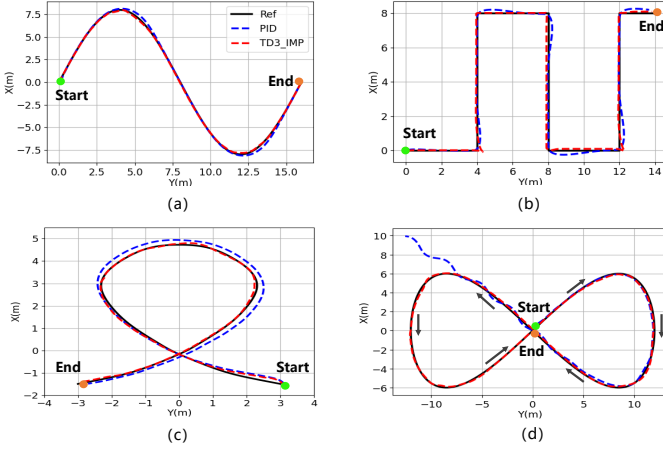


Fig. 12. Trajectories of UUV path-following under different shaped paths using the control policy trained via TD3-IMP and the PID controller. (a) sine wave path. (b) comb scanning path. (c) closed curve path. (d) eight-shaped path.

which contain a sine wave, a comb scanning path, a curved closed path, and an eight-shaped path. A fine-tuned PID controller is designed with the parameter of $k_p = 2$, $k_i = 0.1$, $k_d = 0.05$ which exhibits the best performance for the sine wave path, as demonstrated in Fig. 12(a). Then this set of PID parameters is applied to the other paths, as shown in (b), (c), (d).

From Fig. 12, it is observed that both the PID controller and the TD3-IMP controller can generalize well to simple path-following tasks in (a) and (b). However, for more complicated paths (c) and (d), the performance of the PID controller shows an obvious degradation, where overshoots are exhibited in (c) and being unable to finish the task of (d). In contrast, the TD3-IMP controller performed well under all scenarios, highlighting a significant advantage over PID controller. Here, the MAE (Mean Absolute Error) is used as the performance index to compare the tracking error of each scenario. Their corresponding tracking errors are illustrated in Table III. As shown in Table III, there is a significant increase in PID error corresponding to the rise in trajectory complexity. Furthermore, it can also be observed that the MAE value of TD3-IMP under all tracking scenarios is apparently lower than that of PID. These results indicate that our TD3-IMP controller trained under the 3D environment can be effectively generalized to 2D path-following tasks, and shows greater adaptability under complex tracking scenarios than the traditional PID controller.

2) *3D path-following under modeling uncertainties*: To further verify the effectiveness of the TD3-IMP controller in 3D path-following control tasks, the vehicle is required to follow a helix path. The equation of the helix path is defined as $x_r = 5 - 5 \cos(\varsigma)$, $y_r = 5 \sin(\varsigma)$, $z_r = -2\varsigma - 1$, where $\varsigma \in [0, 4\pi]$. The initial position of the vehicle is $(x_0, y_0, z_0, \psi_0) = (0, -0.1, 0, 0)$. The traditional PID controller is also tested for comparison where its parameters are set as $k_p = 3$, $k_i = 0.05$, $k_d = 0.01$. The hydrodynamic parameters are randomly selected in the range of 0.8~1.2

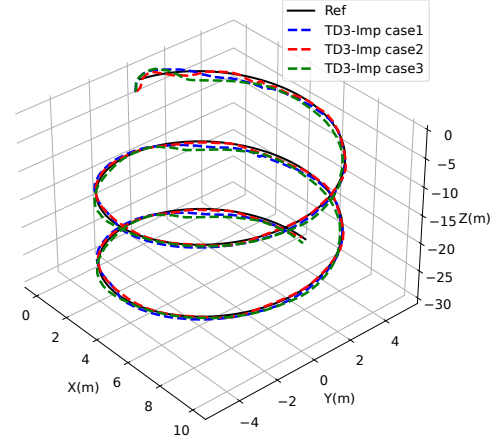


Fig. 13. 3D path-following of TD3-IMP controller under modeling uncertainty.

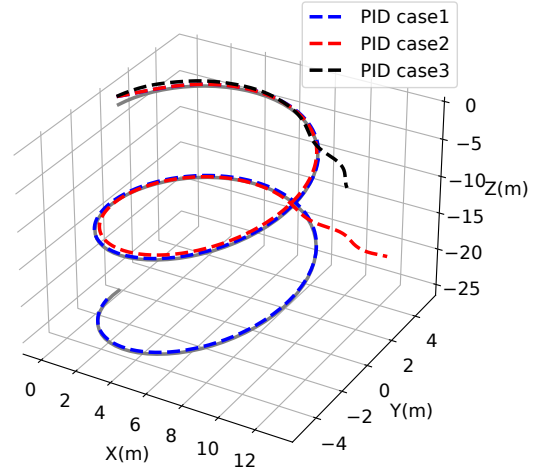


Fig. 14. 3D path-following of PID controller under modeling uncertainty.

times with the original parameters. The simulation is repeated for 15 times for both the PID controller and the TD3-IMP controller.

Fig. 13 and Fig. 14 present the trajectory results of three trials for both TD3-IMP and PID controllers. As observed, the PID controller struggles to track the desired path in two instances, whereas the TD3-IMP controller consistently succeeds under uncertainty settings. The failed trajectories corresponding to these instances are depicted in Fig. 14. Fig. 15 illustrates the average tracking errors—both horizon and depth errors—of the PID and TD3-IMP controllers. The respective variances of these errors are detailed in Table IV. Despite the variance of the two controllers being comparable, the PID controller failed twice during this experiment. This suggests that the TD3-IMP controller provides greater robustness and stability when handling modeling uncertainties.

3) *3D path-following under external disturbances*: The UUV's tracking performance under external disturbances is tested in this subsection. The settings of the underwater vehicle 3D path-following control are the same as previous. The disturbance added to the vehicle is supposed as $\tau_{\Delta x} = 6 \sin(0.1t)N$,

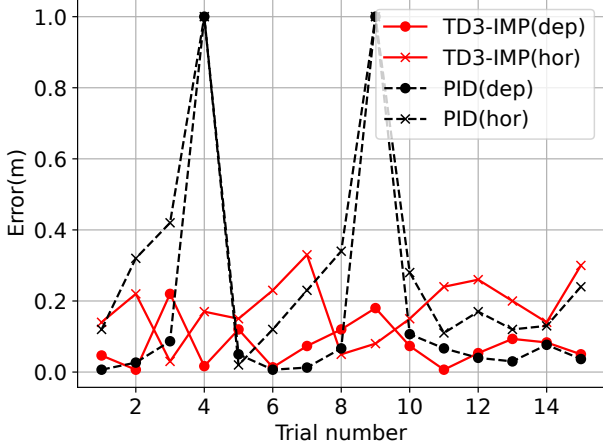


Fig. 15. The variance results of the TD3-IMP and the PID under 15 tests. The value of the error will be set as 1 if the trial fails.

TABLE IV
THE TRACKING VARIANCE OF DEPTH AND HORIZON UNDER THE CONTROLLER OF PID AND RL.

	PID(dep)	TD3-IMP(dep)	PID(hor)	TD3-IMP(hor)
Var	0.0012	0.0037	0.0115	0.0092

$\tau_{\Delta y} = 5 \cos(0.2t)N$, $\tau_{\Delta z} = 4 \cos(0.3t)N$. The obtained results of the 3D path-following control under the external disturbances are depicted in Fig. 16-Fig. 18. It can be noted from Fig. 16 that the performance of the PID controller exhibits a noticeable oscillation, while the performance under the TD3-IMP controller is rarely influenced, presenting a smoother tracking trajectory. Fig. 17 and Fig. 18 demonstrate their corresponding tracking errors in horizontal and depth directions. The horizontal error of the PID controller presents strong oscillation while that of the TD3-IMP controller is significantly reduced, highlighting that the TD3-IMP controller achieves superior performance in the presence of external disturbances.

D. Experiments validation

In this section, experiments are presented to verify the effectiveness of our proposed method. The experiment platform utilized in our research is the BlueRov2, a commercial UUV developed by BlueRobotics. The UUV is equipped with an Inertial Measurement Unit (IMU) for providing angular velocity and acceleration data, and STM32 microprocessors for generating control commands. Additionally, a Raspberry Pi is integrated onboard, utilizing the MAVLink protocol to facilitate communication between the UUV and the host computer. The experiments are done in a lake with a depth exceeding 30m. Given the precision limitations of the underwater module, we employ Real Time Kinematics (RTK) equipment as the localization system, enabling precise centimeter-level positioning and acquisition of the UUV's location and velocity information. As GPS signals are unavailable underwater, our

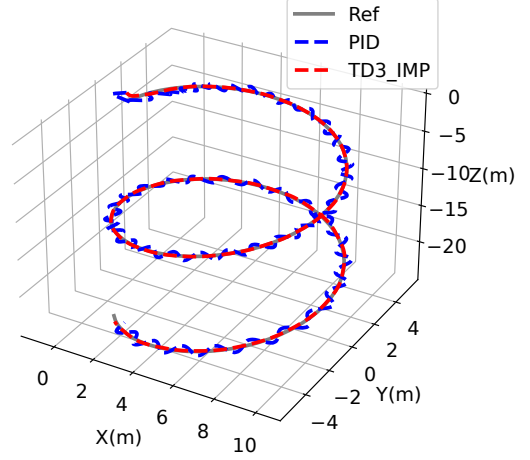


Fig. 16. Trajectory of TD3-IMP controller and PID controller under external disturbances.

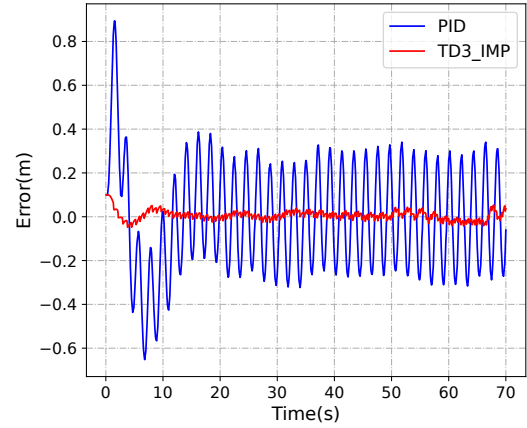


Fig. 17. Horizon error of TD3-IMP controller and PID controller under external disturbances.

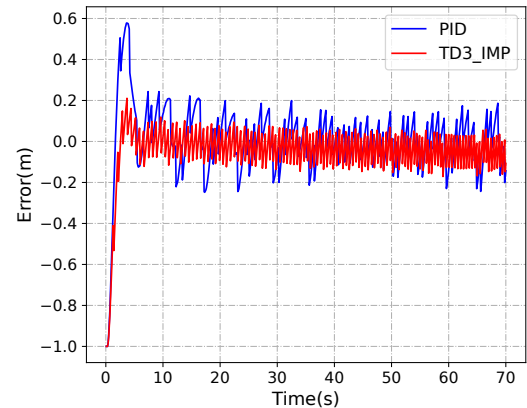


Fig. 18. Depth error of TD3-IMP controller and PID controller under external disturbances.

3D path-following experiments are divided into water surface tracking experiments and depth tracking experiments. Sensor measurements are fused using an extended Kalman filter for enhanced accuracy. A picture of the running experiment on the testing field is shown in Fig. 19.

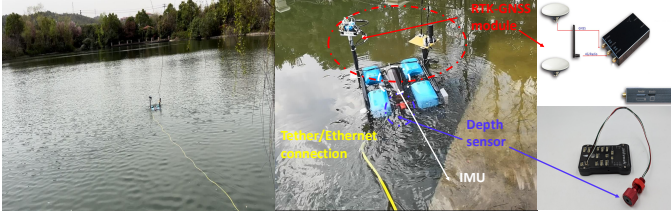


Fig. 19. Deployment of the underwater experimental platform.

To mitigate the gap between simulation and real-world conditions, before implementing the policy trained from simulation to the real-time experiment, we drive the UUV to collect real-time data in the water environment. Then these data will be merged with the existing simulation data in the experience replay buffer and the neural networks will be retrained. Following the training process as detailed in the paper, the weights of the actor-network will be saved and uploaded for the path-following control experiment.

We first conduct experiments of depth tracking, where the UUV is required to track the depth of -6m, -10m, and -20m. The corresponding trajectory results are depicted in Fig. 20. It can be seen that our proposed TD3-IMP controller can successfully achieve depth tracking under different values. Additionally, there are minor fluctuations observed in the descent curve. This is attributed to the UUV colliding with some underwater rocks during the descent process. The MAE error of depth is presented in Table V, showing a comparatively small tracking error. These results indicate the effectiveness of our proposed controller and its applicability to real-world depth control scenarios.

The second experiment is the water surface path-following task. The vehicle is required to follow a path composed of 5 straight lines and 5 waypoints, which are located at (0,0), (10,0), (10,15), (20,15), (30,0). We set two pairs of experiments, the first one is conducted in a minor windy environment (wind level 2), and the second one is tested in a windy environment (wind level 5), where the wave introduced by wind can be seen as the main external disturbance to the UUV. For comparative analysis, the PID controller was employed. The trajectory performance of both controllers is depicted in Fig. 21, Fig. 22, and their tracking error is evaluated in Table VI. As the results illustrate, the TD3-IMP controller and the PID controller achieved a comparable performance when the disturbance was minor. However, for the second experiment, though the tracking performance of TD3-IMP shows a slight oscillation from the reference path due to the irregular and unpredictable wave disturbances, its overall performance is satisfactory where the tracking error remained in an acceptable range of 0.132m, almost half of that of the PID controller. These results indicate that our proposed controller can be

TABLE V
MAE ERROR OF TD3-IMP DEPTH CONTROL EXPERIMENTS

Depth(m)	-6	-10	-20
Error(m)	0.02	0.06	0.05

effectively applied to real-world path-following tasks and has good robustness to external disturbances.

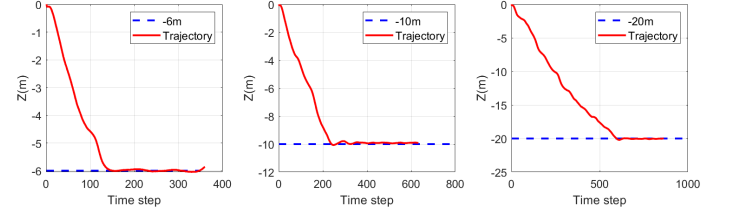


Fig. 20. Trajectories of UUV in tracking depths of -6m, -10m, and -20m under the TD3-IMP controller

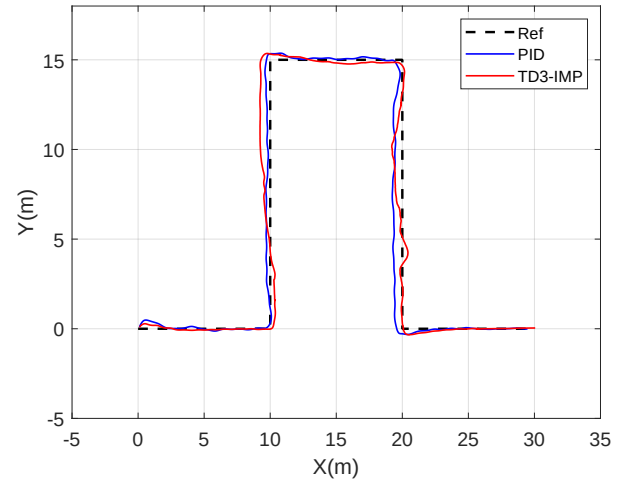


Fig. 21. UUV trajectories of the PID controller and the TD3-IMP controller under minor disturbed condition.

TABLE VI
COMPARISON RESULTS OF MAE FOR PID AND TD3 CONTROLLERS UNDER DISTURBED AND MINOR DISTURBED CONDITIONS.

Controller	Minor disturbance		Under disturbance	
	$e_{xy}(m)$	$e_{\delta}(rad)$	$e_{xy}(m)$	$e_{\delta}(rad)$
PID	0.05	0.1	0.25	0.4
TD3-IMP	0.05	0.1	0.13	0.2

VI. CONCLUSION

In this paper, an improved TD3 algorithm was proposed to realize the path-following control of unmanned underwater vehicles. A newly developed experience replay method was designed to accelerate the training process and realize higher sample utilization. To address the issue of aggressive oscillations in the generated control policies, a new policy

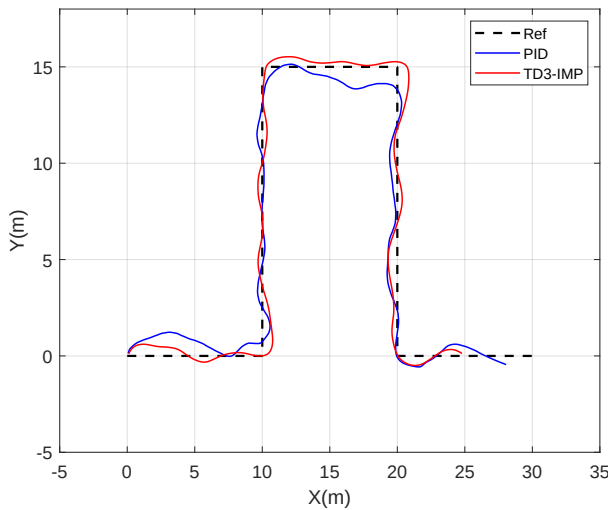


Fig. 22. UUV trajectories of the PID controller and the TD3-IMP controller under disturbed condition.

smoothness method was proposed that dynamically adjusted tuning weights according to the current tracking error. Simulation results showed that the proposed algorithm achieved faster convergence speed and better tracking results than other reinforcement learning agents including DDPG, vanilla TD3, and TD3-PER. Additionally, our algorithm showcased better generalization capabilities across varying path configurations and exhibited superior robustness in handling uncertainties and disturbances, which outperformed the commonly used controller for unmanned underwater vehicles. **The simulations and real-world experiments have demonstrated the superiority of the proposed approach. A limitation of the proposed method lies in its data-driven nature, lacking the complete utilization of physical information or models. Future exploration will involve hybrid methods, aiming to integrate beneficial physical insights with DRL algorithms for UUV control tasks. This can potentially improve the convergence speed of RL, thereby further improving the stability and efficiency of the learning process.**

REFERENCES

- [1] D. L. Rizzini, F. Kallasi, J. Aleotti, F. Oleari, and S. Caselli, "Integration of a stereo vision system into an autonomous underwater vehicle for pipe manipulation tasks," *Computers & Electrical Engineering*, vol. 58, pp. 560–571, 2017.
- [2] S. B. Williams, O. Pizarro, I. Mahon, and M. Johnson-Roberson, "Simultaneous localisation and mapping and dense stereoscopic seafloor reconstruction using an auv," in *Experimental Robotics*. Springer, 2009, pp. 407–416.
- [3] A. Palomer, P. Ridao, and D. Ribas, "Inspection of an underwater structure using point-cloud slam with an auv and a laser scanner," *Journal of Field Robotics*, vol. 36, no. 8, pp. 1333–1344, 2019.
- [4] R. B. Wynn, V. A. Huvenne, T. P. Le Bas, B. J. Murton, D. P. Connelly, B. J. Bett, H. A. Ruhl, K. J. Morris, J. Peakall, D. R. Parsons *et al.*, "Autonomous underwater vehicles (auvs): Their past, present and future contributions to the advancement of marine geoscience," *Marine Geology*, vol. 352, pp. 451–468, 2014.
- [5] C. Shen, Y. Shi, and B. Buckham, "Path-following control of an auv: A multiobjective model predictive control approach," *IEEE Transactions on Control Systems Technology*, vol. 27, no. 3, pp. 1334–1342, 2018.
- [6] J. Guerrero, J. Torres, V. Creuze, and A. Chemori, "Adaptive disturbance observer for trajectory tracking control of underwater vehicles," *Ocean Engineering*, vol. 200, p. 107080, 2020.
- [7] C. Shen, Y. Shi, and B. Buckham, "Trajectory tracking control of an autonomous underwater vehicle using lyapunov-based model predictive control," *IEEE Transactions on Industrial Electronics*, vol. 65, no. 7, pp. 5796–5805, 2017.
- [8] S. Heshmati-Alamdari, G. C. Karras, P. Marantos, and K. J. Kyriakopoulos, "A robust predictive control approach for underwater robotic vehicles," *IEEE Transactions on Control Systems Technology*, vol. 28, no. 6, pp. 2352–2363, 2019.
- [9] C. Yu, X. Xiang, P. A. Wilson, and Q. Zhang, "Guidance-error-based robust fuzzy adaptive control for bottom following of a flight-style auv with saturated actuator dynamics," *IEEE Transactions on Cybernetics*, vol. 50, no. 5, pp. 1887–1899, 2019.
- [10] M. H. Khodayari and S. Balochian, "Modeling and control of autonomous underwater vehicle (auv) in heading and depth attitude via self-adaptive fuzzy pid controller," *Journal of Marine Science and Technology*, vol. 20, no. 3, pp. 559–578, 2015.
- [11] J. Yan, J. Gao, X. Yang, X. Luo, and X. Guan, "Position tracking control of remotely operated underwater vehicles with communication delay," *IEEE Transactions on Control Systems Technology*, vol. 28, no. 6, pp. 2506–2514, 2019.
- [12] L. Liu, L. Zhang, G. Pan, and S. Zhang, "Robust yaw control of autonomous underwater vehicle based on fractional-order pid controller," *Ocean Engineering*, vol. 257, p. 111493, 2022.
- [13] Z. Xu, M. Haroutunian, A. J. Murphy, J. Neasham, and R. Norman, "A comparison of functional control strategies for underwater vehicles: Theories, simulations and experiments," *Ocean Engineering*, vol. 215, p. 107822, 2020.
- [14] S. Soylu, A. A. Proctor, R. P. Podhorodeski, C. Bradley, and B. J. Buckham, "Precise trajectory control for an inspection class rosv," *Ocean Engineering*, vol. 111, pp. 508–523, 2016.
- [15] H. Dong and X. Zhao, "Data-driven wind farm control via multiplayer deep reinforcement learning," *IEEE Transactions on Control Systems Technology*, vol. 31, no. 3, pp. 1468–1475, 2022.
- [16] H. Hua, Y. Fang, X. Zhang, and C. Qian, "A time-optimal trajectory planning strategy for an aircraft with a suspended payload via optimization and learning approaches," *IEEE Transactions on Control Systems Technology*, vol. 30, no. 6, pp. 2333–2343, 2022.
- [17] Y. Zhao, X. Qi, Y. Ma, Z. Li, R. Malekian, and M. A. Sotelo, "Path following optimization for an underactuated usv using smoothly-convergent deep reinforcement learning," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 10, pp. 6208–6220, 2020.
- [18] X. Hu, T. Liu, X. Qi, and M. Barth, "Reinforcement learning for hybrid and plug-in hybrid electric vehicle energy management: Recent advances and prospects," *IEEE Industrial Electronics Magazine*, vol. 13, no. 3, pp. 16–25, 2019.
- [19] H. Yu, S. Park, A. Bayen, S. Moura, and M. Krstic, "Reinforcement learning versus pde backstepping and pi control for congested freeway traffic," *IEEE Transactions on Control Systems Technology*, vol. 30, no. 4, pp. 1595–1611, 2021.
- [20] I. Carlucho, M. De Paula, S. Wang, Y. Petillot, and G. G. Acosta, "Adaptive low-level control of autonomous underwater vehicles using deep reinforcement learning," *Robotics and Autonomous Systems*, vol. 107, pp. 71–86, 2018.
- [21] D. Jiang, J. Huang, Z. Fang, C. Cheng, Q. Sha, B. He, and G. Li, "Generative adversarial interactive imitation learning for path following of autonomous underwater vehicle," *Ocean Engineering*, vol. 260, p. 111971, 2022.
- [22] Y. Fang, S. Huang, J. Pu, and J. Zhang, "Auv position tracking and trajectory control based on fast-deployed deep reinforcement learning method," *Ocean Engineering*, vol. 245, p. 110452, 2022.
- [23] P. Jiang, S. Song, and G. Huang, "Attention-based meta-reinforcement learning for tracking control of auv with time-varying dynamics," *IEEE Transactions on Neural Networks and Learning Systems*, vol. 33, no. 11, pp. 6388–6401, 2021.
- [24] Z. Chu, B. Sun, D. Zhu, M. Zhang, and C. Luo, "Motion control of unmanned underwater vehicles via deep imitation reinforcement learning algorithm," *IET Intelligent Transport Systems*, vol. 14, no. 7, pp. 764–774, 2020.
- [25] S. Fujimoto, H. Hoof, and D. Meger, "Addressing function approximation error in actor-critic methods," in *International conference on machine learning*. PMLR, 2018, pp. 1587–1596.
- [26] H. Wu, S. Song, K. You, and C. Wu, "Depth control of model-free auvs via reinforcement learning," *IEEE Transactions on Systems, Man, and Cybernetics: Systems*, vol. 49, no. 12, pp. 2499–2510, 2018.
- [27] Y. Sun, X. Ran, G. Zhang, X. Wang, and H. Xu, "Auv path following controlled by modified deep deterministic policy gradient," *Ocean Engineering*, vol. 210, p. 107360, 2020.

- [28] C. Zhang, P. Cheng, B. Du, B. Dong, and W. Zhang, "Auv path tracking with real-time obstacle avoidance via reinforcement learning under adaptive constraints," *Ocean Engineering*, vol. 256, p. 111453, 2022.
- [29] C. Chen, H. Tang, J. Hao, W. Liu, and Z. Meng, "Addressing action oscillations through learning policy inertia," in *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 8, 2021, pp. 7020–7027.
- [30] A. Raffin, J. Kober, and F. Stulp, "Smooth exploration for robotic reinforcement learning," in *Conference on Robot Learning*. PMLR, 2022, pp. 1634–1644.
- [31] D. Song, W. Gan, P. Yao, W. Zang, Z. Zhang, and X. Qu, "Guidance and control of autonomous surface underwater vehicles for target tracking in ocean environment by deep reinforcement learning," *Ocean Engineering*, vol. 250, p. 110947, 2022.
- [32] J. Woo, C. Yu, and N. Kim, "Deep reinforcement learning-based controller for path following of an unmanned surface vehicle," *Ocean Engineering*, vol. 183, pp. 155–166, 2019.
- [33] A. Ilyas, L. Engstrom, S. Santurkar, D. Tsipras, F. Janoos, L. Rudolph, and A. Madry, "A closer look at deep policy gradients," *arXiv preprint arXiv:1811.02553*, 2018.
- [34] P. Cai, X. Mei, L. Tai, Y. Sun, and M. Liu, "High-speed autonomous drifting with deep reinforcement learning," *IEEE Robotics and Automation Letters*, vol. 5, no. 2, pp. 1247–1254, 2020.
- [35] S. Mysore, B. Mabsout, R. Mancuso, and K. Saenko, "Regularizing action policies for smooth control with reinforcement learning," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 1810–1816.
- [36] T. SNAME, "Nomenclature for treating the motion of a submerged body through a fluid," *The Society of Naval Architects and Marine Engineers, Technical and Research Bulletin*, no. 1950, pp. 1–5, 1950.
- [37] M. Breivik and T. I. Fossen, "Guidance laws for autonomous underwater vehicles," *Underwater vehicles*, vol. 4, pp. 51–76, 2009.
- [38] G. Antonelli, S. Chiaverini, N. Sarkar, and M. West, "Adaptive control of an autonomous underwater vehicle: experimental results on odin," *IEEE Transactions on Control Systems Technology*, vol. 9, no. 5, pp. 756–765, 2001.
- [39] K. L. Walker, R. Gabl, S. Aracri, Y. Cao, A. A. Stokes, A. Kiprakis, and F. Giorgio-Serchi, "Experimental validation of unsteady wave induced loads on a stationary remotely operated vehicle," in *2021 IEEE international conference on robotics and automation (ICRA)*. IEEE, 2021, pp. 2242–2248.
- [40] C.-J. Wu, "6-dof modelling and control of a remotely operated vehicle," Ph.D. dissertation, Flinders University, College of Science and Engineering., 2018.
- [41] I. J. Goodfellow, J. Shlens, and C. Szegedy, "Explaining and harnessing adversarial examples," *arXiv preprint arXiv:1412.6572*, 2014.
- [42] A. Kurakin, I. J. Goodfellow, and S. Bengio, "Adversarial examples in the physical world," in *Artificial intelligence safety and security*. Chapman and Hall/CRC, 2018, pp. 99–112.
- [43] M. von Benzon, F. Sørensen, J. Liniger, S. Pedersen, S. Klemmensen, and K. Schmidt, "Integral sliding mode control for a marine growth removing rov with water jet disturbance," in *2021 European Control Conference (ECC)*, 2021, pp. 2265–2270.
- [44] T. Schaul, J. Quan, I. Antonoglou, and D. Silver, "Prioritized experience replay," *arXiv preprint arXiv:1511.05952*, 2015.
- [45] Y. Zhong, Y. Yang, K. He, and C. Chen, "Fast terminal sliding-mode control based on unknown input observer for the tracking control of underwater vehicles," *Ocean Engineering*, vol. 264, p. 112480, 2022.